

Graph neural networks (GNNs)

The GNN model is developed based on the deep-learning framework PyTorch [64] and its geometric extension library PyTorch Geometric [65]. The GNN architecture is depicted in Figure 1(b). First of all, the input graph is sent to the input block which uses a combination of PNA convolutional (PNAConv) layer, Gated Recurrent Unit cell (GRUCell), and Batch Normalization layer (BatchNorm) to upscale the dimension of node features to 50. Then the graph is passed to the message passing block which contains 10-time repetition of the combined layers. Within the block, nodes communicate with each other by passing the messages given the node features plus connectivity and update their own node features considering the received messages. The last MLP that returns the predicted Peierls barrier or potential energy change has a structure of an input layer of size 30, a hidden layer of size 20, another hidden layer of size 10, and an output layer of one neuron. ReLU is adopted as the activation function in this MLP. We adopt default weight and bias initialization of all of the layers in the model defined by PyTorch Geometric.

Model training and evaluation

All the datasets created for the training/validation compositions (see Figure S1) are split into the train set (90% data) and validation set (10% data). The models were trained with a batch size of 32 using the Adam optimization method [66] for 250 epochs on one NVIDIA Tesla V100s with 32GB memory. Training starts with a learning rate of 0.0005, and a dynamic learning rate scheduler named ReduceLROnPlateau reduces the learning rate by half if no improvement is seen for 10 epochs to minimize the validation MSE. The learning curves of the GNNs are shown in Fig. S4 in the SI which indicate the convergence of training for both models.

Solute-strengthening theory in BCC alloys

We apply Maresca-Curtin screw strengthening theory for non-dilute to high-entropy alloys to compute the yields stress [24]. The theory is established based on the assumption that the initially straight dislocation becomes spontaneously kinked at zero load and zero temperature so as to lower their total energy. Three mechanisms contribute to the screw dislocation strengthening; (I) Peierls-like mechanism, (II) kink glide mechanism, and (III) cross-kink formation and unpinning and the alloy strength at temperature T is expressed by

$$\tau(\dot{\epsilon}, T) = \tau_{xk}(\dot{\epsilon}, T) + \min[\tau_k(\dot{\epsilon}, T), \tau_p(\dot{\epsilon}, T)] \quad (2)$$

where $\dot{\epsilon}$ is the experimental strain rate and τ_p , τ_k , and τ_{xk} are the Peierls strength, kink migration strength, and cross-kink unpinning strength, respectively. [24, 21]

Agent design

We design AI agents using the state-of-the-art all-purpose LLM GPT-4 and dynamic multi-agent collaboration is implemented in AutoGen framework[67], an open-source ecosystem for agent-based AI modeling. Additional agents are introduced as described below.

In our multi-agent system, the human *user* agent is constructed using UserProxyAgent class from Autogen, and *Assistant*, *Planner*, *Reviewer*, *coder* agents are created via AssistantAgent class from Autogen, while *multi-modal* agent is constructed via MultimodalConversableAgent class. Each agent is assigned a role through a profile description as shown in Figures S8-S12 in the Supplementary Material, included as *system_message* at their creation.

Function and tool design

All the tools implemented in this work are defined as python functions. Each function is characterized by a name, a description, and input properties. The full list of tools and their descriptions can be found in the corresponding codes.

Data and code availability

All data and codes are available on GitHub at <https://github.com/lamm-mit/AlloyAgents>. Alternatively, they will be provided by the corresponding author based on reasonable request.

Author Contributions: M.J.B and A.G. conceived the overall concept. A.G and M.J.B developed the GNN model and multi-agent system. A.G. curated the training and testing data for the GNN